

CLI help and usability research for herdr-threads

CLI help and usability research for herdr-threads

Executive Summary

Introduction and method

Main Analysis

Finding 1: A usage grammar is not a purpose statement

Finding 2: Explain options, defaults, and constraints where they are used

Finding 3: Examples should show the task and the next safe step

Finding 4: Output format is a local presentation choice for recovery results

Finding 5: Compatibility and accessibility are part of help quality

Synthesis and recommendations

Limitations and caveats

Bibliography

Methodology appendix

CLI help and usability research for herdr-threads

Executive Summary

The reported service help gap is representative of the command tree. At baseline 0d70765e, `herdr-threads service --help` began with `Usage: herdr-threads service [OPTIONS] <COMMAND>` and gave empty summaries for `inspect` and `disconnect`. The root help likewise listed many public commands with no descriptions. A caller had to guess whether `service inspect` inspected a process, connection, thread service, or agent. An effective group help page should state its purpose, summarize each action, and show one workflow that can be copied after replacing named placeholders. The Command Line Interface Guidelines explicitly recommend a description, example invocations, flag explanations, and discoverable subcommand help; Microsoft's command line guidance treats a grouping command as an area that organizes related actions. [1][2]

The `service inspect` error is a separate real defect. The CLI defaults to text output. The daemon's recovery route accepts only its default wire output specification, which is JSON with no continuation context. The ordinary CLI path always sent the selected output specification, so a normal `service inspect` request was rejected with `selected output is unavailable for service recovery (unsupported)` before the daemon ran the inspection. The typed recovery response is already serializable, and the CLI already encodes it as text or JSON. The narrow repair is to send service recovery commands through the existing default wire request and render the returned result in the caller's selected format locally. This respects the daemon's protocol restriction and does not assert an unsupported server feature. The same route covers `service disconnect`; its boot and generation preconditions remain enforced by the daemon.

The highest value help changes are purpose lines for public commands, a service recovery example, required-value descriptions, message input choices, and accurate page defaults. They leave command names, grammar, output shapes, and exit codes intact. The existing `--json`, `--human`, and `--machine` selection deserves explanation because terminal presentation and script output have different consumers. Official AWS and Azure references document human and machine formats, while the CLI Guidelines distinguish result data on `stdout` from diagnostic messaging on `stderr`. [1][7][8]

This report is an evidence-based audit of the current public help and the specific `service inspect` failure. It is not a recommendation to rename commands, add new output formats, or change seat authority. Doctor-specific parser, output, debug, and fix work is assigned to the separate doctor-redesign branch. Its research is noted below for coordination, not implemented here.

Introduction and method

The research question was: what should a multi-command CLI's help say so a person can find and safely complete a task, and how does that apply to herdr-threads? The primary sources were the CLI Guidelines, documentation from the maintainers of `clap` and `Python argparse`, Microsoft's command-line design guidance, Google's documentation and accessibility style guides, and official GitHub, AWS, and Azure CLI references. GNU coding standards supplied a conventional `--help` baseline. Sources were selected for direct design or implementation guidance rather than commentary about other products. [1][2][3][4][5][6][7]

[8][9][10][11]

The local audit inspected the clap parser in `src/cli/commands.rs`, the terminal endpoint in `src/main.rs`, the selected encoder in `src/protocol/output.rs`, the wire read in `src/cli/mod.rs`, the recovery gate in `src/daemon/transport.rs`, and the command result types. It also ran the built binary's root, service, thread, seat, and daemon help. This provides direct evidence about this repository rather than assuming that a framework's advertised features appear in our generated help. The audit did not contact or restart the user's daemon. [12]

The recommendations favor precise wording and a minimal service request repair. Herdr-threads is both human-facing and automation-facing, so the research evaluates discoverability and script compatibility together. A CLI documentation convention is guidance, not a requirement to copy every pattern. In particular, the current product uses `--machine` as a stable key: `value` representation and `--json` for structured output; replacing that contract merely to resemble another CLI would impose compatibility cost. Microsoft explicitly warns that a CLI is hard to change once scripts rely on it. [2]

Main Analysis

Finding 1: A usage grammar is not a purpose statement

Usage: `herdr-threads service [OPTIONS] <COMMAND>` describes syntax only. It does not say that the group observes and, under explicit preconditions, disconnects the daemon's live service connection. At baseline the `Commands:` section showed `inspect` and `disconnect` with empty descriptions. The same issue appeared in the `thread`, `seat`, and `daemon` group pages. At the root, actions such as `inbox`, `read`, `send`, `ack`, `search`, and `service` also had empty summaries, while `setup` had a much longer multi-sentence summary. This inconsistent density makes scanning the help harder: the names that new users need first carry the least explanation. [12]

The CLI Guidelines' concise-help recipe is a description, one or two examples, flag descriptions, and a path to full help. Its discoverability section links examples and next-step suggestions to lower recall burden. Microsoft's guidance explains that group commands organize related subcommands. GitHub's official CLI reference shows a related navigational pattern: a user can ask for a command group or append `--help` to a nested command. Those sources converge on a small information hierarchy: a group-level purpose, action-level summaries, and detailed action help. [1][2][6]

Clap supports that hierarchy directly. Its derive documentation maps `command_about` and `long_about` to command descriptions, and `Arg::help` and `long_help` to parameter descriptions. Doc comments are a supported way to supply those values. A parser change is therefore unnecessary for most of this audit: short doc comments on the existing variants and arguments improve generated help while keeping grammar and dispatch unchanged. Python `argparse`'s `description` and `epilog` roles provide an independent example of separating the command's purpose from material after the option list. [3][11]

For `herdr-threads`, the service group should say "Inspect or disconnect the live service connection for recovery." `inspect` should state that it observes the live connection, boot and generation, and does not attest native agent liveness. `disconnect` should state that it revokes one observed connection and does not stop the daemon. The distinction matters because `daemon stop` is a different action, and recovery results explicitly represent a connection rather than general process health. [12]

The root help should prioritize one-line descriptions that let a reader distinguish the daily loop: `inbox` finds attention, `read` observes messages without ACK, `send` publishes a message, and `ack` records exact message IDs. A short example sequence in root help gives a route into the full grammar. The examples use placeholders `THREAD` and `MESSAGE` whose meaning follows from the corresponding action help. Google's syntax guide recommends explaining placeholders and removing optional syntax from copyable examples when it obscures the real invocation. [1][4]

Finding 2: Explain options, defaults, and constraints where they are used

Generated help at baseline included `--json` with no text and `service disconnect --expected-boot` and `--expected-generation` without explanations. The latter are safety preconditions: the operator must pass a boot and connection generation observed from `service inspect`. Without that explanation, a syntactically valid flag list does not tell an operator how to obtain its values or why stale values fail. The daemon checks both and returns a typed stale-generation error when the observed state is no longer active. [12]

Argument descriptions should answer what a value is, where it comes from, and what changes when it is supplied. Google's syntax and placeholder guidance explicitly asks documentation to explain placeholder meanings. The CLI Guidelines recommend describing flags in concise help and providing practical error recovery. Clap exposes a distinct help value for each argument, so the wording can live beside its parser declaration. For `service disconnect`, an example using `BOOT` and `GENERATION` plus a sentence naming the `inspect` fields closes the workflow without adding a command or relaxing a guard. [1][3][4]

Defaults matter when omission changes the amount or presentation of data. Python argparse offers a formatter that automatically appends defaults to argument help, illustrating the discoverability value. Herdr-threads uses `PageRequest` defaults of 20 items and 16,384 encoded bytes, and `read --follow` starts with 20 recent messages unless a different initial range is selected. Its output presentation defaults to human text on an ordinary terminal and machine text when stdout is not a terminal or an agent harness is detected. Stating these actual defaults in help is useful; stating an inferred or stale value would be worse than leaving it blank. [1][12]

The same principle applies to `send: --body, --file, and --stdin` are mutually exclusive input choices; `--require-ack` names recipients, and `--deadline` is parsed in positive seconds. A reader should see these meanings in `send --help`. The parser already enforces the body-source group and rejects duplicate explicit recipients, so these descriptions do not create new behavior. The help text should avoid a claim that every send requires ACK, because `--require-ack` is optional. [12]

Clarity also requires restraint. Four cooperative-context flags are exposed as global options, but their exact combination is part of a specialized agent path. Short descriptions can identify the seat, target, harness, and role and say that all four must be supplied together. A future pass could move them to an advanced section if compatibility permits. This report does not recommend hiding or changing them now: the concrete complaint concerns absent purpose and the recovery failure, and speculative help restructuring would enlarge the review surface. [1][12]

Finding 3: Examples should show the task and the next safe step

An example that merely repeats `Usage:` adds little. A service recovery example has two steps: inspect, then disconnect with the returned boot and generation. It communicates an order of operations and identifies the exact values needed for an irreversible connection revocation. The help also needs to say that a stale boot or generation is refused, since that is a meaningful recovery outcome. The CLI Guidelines recommend examples early in help and errors that suggest a correction or next action. GitHub's CLI reference demonstrates help at each level of a command tree. [1][6]

At the root, examples of `inbox`, `read THREAD`, `send THREAD --body 'Hello'`, and `ack MESSAGE` map to the ordinary user loop. Their placement after options is a clap `after_help` tradeoff: the parser makes it easy to add examples without replacing the full help template. That preserves the parser's generated usage and flag list. For a later documentation pass, a task-oriented short-help template could move examples higher, but that requires checking all nested help for consistency and terminal width; it is not necessary to resolve the present gap. [1][3][12]

Examples need compatibility review. They should use documented commands and flags, avoid shell constructs that may execute unexpectedly, and not imply that a read ACKs or that a service inspection proves agent liveness. A single example with named placeholders is easier to maintain than a transcript of dynamic IDs or state. Google's command-line style guidance distinguishes input commands from output and asks for placeholder explanation. Its accessibility guidance recommends short direct sentences and putting distinguishing information first, which helps a reader scan help in a narrow terminal or through assistive technology. [4][5]

Doctor-specific examples belong in doctor-redesign. In coordination, that branch reported that other CLIs distinguish concise defaults from verbose or debug checks and make `--fix` report the repairs attempted. That is a useful comparison for doctor help, but an implementation must describe doctor's actual checks and bounded repairs, not inherit another CLI's promises. The doctor branch owns its parser, output, debug, and fix semantics. [12]

Finding 4: Output format is a local presentation choice for recovery results

The observed `service inspect` failure is not malformed syntax. The parser accepts it as a typed `ServiceInspect` command. The CLI's parsed output defaults to `Text`; `--json` selects `Json`, and context selectors may be present. The daemon's service-control branch compares the request output against `OutputSpec::default()`, which is `Json` with empty context, and emits `Unsupported` on any difference. Consequently the ordinary text invocation and JSON plus an explicit context both fail at the recovery gate. A transport test demonstrates that a default recovery request can return a typed `ServiceInspection`, so the missing operation is not inspection itself. [12]

The service result has no server-generated continuation command. The CLI's selected encoder already serializes any `CommandResult` as JSON and has a text fallback for result objects. Thus the selected format can be applied after the typed result returns. The bounded repair uses `LocalClient::call`, whose transport request omits selected output, only for `ServiceInspect` and `ServiceDisconnect`; the existing `write_selected` path then produces text or JSON locally. This is an inference from, and a change to, the local code path. It does not require the daemon to accept selected recovery output or change its authority and generation checks. [12]

AWS and Azure both document distinct machine and human-friendly output forms and the role of an explicit output selector. Their exact format lists differ from this product's and are not a reason to add YAML or TSV. The relevant design principle is that a selected

presentation should reliably affect a successful command's result. Here, a local typed result supports that principle without enlarging the recovery wire contract. AWS additionally documents that machine formats may have ordering hazards, which reinforces preserving the existing stable key: `value` and JSON contracts rather than treating a pretty table as a script API. [7][8][9]

The current error taxonomy is already useful: an unsupported capability is exit 4, while invalid arguments are exit 2, unavailable daemon or host is exit 3, and an unknown mutation outcome is exit 5. The reported error was correctly typed for the daemon branch it reached but misleading for a normal command that had a supported local rendering path. Removing the accidental rejection is better than changing the error text or reclassifying it. The CLI Guidelines and AWS CLI reference both emphasize meaningful return codes; the fix preserves those distinctions for real failures. [1][10][12]

Finding 5: Compatibility and accessibility are part of help quality

The current entrypoint prints successful requested help to stdout and usage for a bare invocation to stderr with exit 2. Ordinary result output goes to stdout; error display is written to stderr. These choices follow the CLI Guidelines' stream separation and exit-status convention. The top-level help also documents the product's status map, including the specific action for an unknown outcome. The audit found no reason to change those contracts to solve the reported problems. [1][12]

Large multi-command CLIs create two competing needs: overview and precision. Short group summaries help discover a command. Focused action help explains a value or safety constraint. The root page should remain scannable: one-line action descriptions fit better there than long implementation details, while `service disconnect --help` can explain the observed-value workflow. Google's accessibility guide recommends direct language, short sentences, descriptive headings, and meaningful links. Applying those recommendations to terminal help means making the first phrase identify the action and avoiding unexplained jargon where a plain term exists. [5]

Compatibility limits speculative cleanup. The top-level `--json`, `--human`, and `--machine` flags, the stable machine output, result envelope, command spellings, and exit statuses may be used by scripts. Microsoft specifically cautions that changing an established CLI can break those scripts. This report recommends help copy and a narrow request-path correction, with tests against the generated help and selected recovery output. It does not propose a new hierarchy, renamed actions, or a broad error renderer rewrite. [2][12]

Synthesis and recommendations

The sources agree on a practical order: explain purpose, show the common action, describe required input, and give recovery guidance when a command can fail. The repository's observed gap is concentrated in missing about text despite clap's ability to generate a useful hierarchy. The `service inspect` report exposed a deeper mismatch between a global output selector and a special recovery route. The two issues meet at the same workflow: help should teach the recovery operation, and the normal inspection invocation must actually work. [1][2][3][12]

First, retain one-line purpose statements on every public root action and on key nested actions. Verify the `service`, `thread`, `seat`, `daemon`, and root help as rendered, because source comments alone cannot prove the final help is readable. Second, document actual defaults and value origins for page flags, message input, and service recovery preconditions. Third, keep a short root example sequence and a two-step service recovery example. Fourth, send service recovery commands with the default wire request and encode the typed result locally; keep the daemon restriction and explicit disconnect guards. Finally, integrate doctor help only after the doctor branch settles its actual behavior, so its documentation reflects supported debug and repair operations. [1][3][4][12]

Limitations and caveats

The study reviews generated help and selected local code paths, not a usability study with new users. The service defect was reproduced with a fake client implementing the daemon's documented rejection and traced through the actual daemon guard; it did not invoke the user's shared daemon. A private end-to-end recovery run would strengthen confidence but would also require a private service instance, so the focused regression and existing transport tests are the right first checks. The output fix applies only to recovery commands whose results do not contain server-generated continuations. Future recovery commands with continuation fields would need a separate review. [12]

The cited official CLI references provide examples of good documentation and output contracts, not a single universal standard. Their defaults and flags differ. This report transfers the principle of clear selection and stable script output, not another product's exact syntax. The deep-research skill's default artifact directory under `~/Documents` is outside this task's writable workspace, so the evidence and report are kept in this branch under `docs/research/2026-10-03-cli-help/` for review and integration.

Bibliography

[1] Prasad, Firshman, Tashian, and Parish (accessed 2026-10-03). “Command Line Interface Guidelines.” <https://clig.dev/> [2] Microsoft (accessed 2026-10-03). “Command-line design guidance for System.CommandLine.” <https://learn.microsoft.com/en-us/dotnet/standard/commandline/design-guidance> [3] clap maintainers (accessed 2026-10-03). “clap derive documentation.” https://docs.rs/clap/latest/clap/_derive/ [4] Google (accessed 2026-10-03). “Document command-line syntax.” <https://developers.google.com/style/code-syntax> [5] Google (accessed 2026-10-03). “Write accessible documentation.” <https://developers.google.com/style/accessibility> [6] GitHub (accessed 2026-10-03). “GitHub CLI reference.” <https://docs.github.com/en/github-cli/github-cli/github-cli-reference> [7] Amazon Web Services (accessed 2026-10-03). “Setting the output format in the AWS CLI.” <https://docs.aws.amazon.com/cli/latest/userguide/cli-usage-output-format.html> [8] Microsoft (accessed 2026-10-03). “Output formats for Azure CLI commands.” <https://learn.microsoft.com/en-us/cli/azure/format-output-azure-cli> [9] Microsoft (accessed 2026-10-03). “Query Azure CLI command results.” <https://learn.microsoft.com/en-us/cli/azure/use-azure-cli-successfully-query> [10] Amazon Web Services (accessed 2026-10-03). “Command line return codes in the AWS CLI.” <https://docs.aws.amazon.com/cli/latest/userguide/cli-usage-returncodes.html> [11] Python Software Foundation (accessed 2026-10-03). “argparse — Parser for command-line options, arguments and sub-commands.” <https://docs.python.org/3/library/argparse.html> [12] herdr-threads (baseline 0d70765e, audited 2026-10-03). `src/cli/commands.rs`, `src/cli/mod.rs`, `src/cli/output.rs`, `src/protocol/output.rs`, `src/daemon/transport.rs`, `src/main.rs`; generated `--help` and focused tests in this branch.

Methodology appendix

The search plan covered purpose statements, nested help, examples, placeholders, default values, errors, stdout/stderr, exit statuses, human and machine formats, compatibility, and accessibility. Results were limited to primary official documentation and directly inspected project files. Every external source has a canonical URL in `sources.jsonl`; excerpts and locators are persisted in `evidence.jsonl`; the principal conclusions are in `claims.jsonl`. Claims about this CLI are grounded in source files and baseline command output, not transferred from the external examples. Source agreement was strongest for purpose/help, examples, and output/error separation. The main local finding was the recovery route’s default-output gate, which shifted the implementation from changing help alone to correcting the client request path.

The report was checked for source support and for counterexamples: adding a new server-selected output capability was considered and rejected because the current recovery route explicitly does not accept one; changing only the error message was rejected because it would leave the normal invocation unusable. Doctor behavior was kept in its separately owned branch. No user configuration, shared Herdr server, or real daemon was changed during research.